

Progetto - Decentralised Lending Service

Progetto — Decentralised Lending Service

Panoramica ad Alto Livello

Il progetto richiede di implementare un **servizio di prestito decentralizzato** su Ethereum. L'idea fondamentale è quella di una piattaforma di pooling collettivo: i contributor depositano ETH in un fondo comune, i richiedenti (applicant) possono chiedere prestiti da quel fondo, e la comunità dei contributor vota democraticamente (con peso proporzionale al capitale non impegnato) se approvare o rifiutare ogni proposta. Come garanzia di solvibilità, il sistema usa un **oracolo Bitcoin** che legge i saldi degli indirizzi BTC dalla blockchain Bitcoin reale e li espone su Ethereum.

Il sistema si compone di **tre grandi blocchi**:

1. **Smart contracts Ethereum** — il cuore della logica decentralizzata (pool, prestiti, voto, rimborsi, compensazioni, upgradeability).
2. **Oracolo Bitcoin** — un componente ibrido: uno smart contract Ethereum che memorizza i saldi BTC, e uno script Python off-chain che legge i primi 131 000 blocchi della mainnet Bitcoin per calcolare gli UTXO e aggiornare il contratto.
3. **Script Python** — per il setup iniziale, la demo, il bot del contributor e la misurazione del gas.

In una frase

Devi costruire un protocollo DeFi di lending su Ethereum con governance on-chain a voto ponderato, un oracolo cross-chain BTC→ETH, e una suite completa di script Python e test Hardhat.

Architettura del Sistema

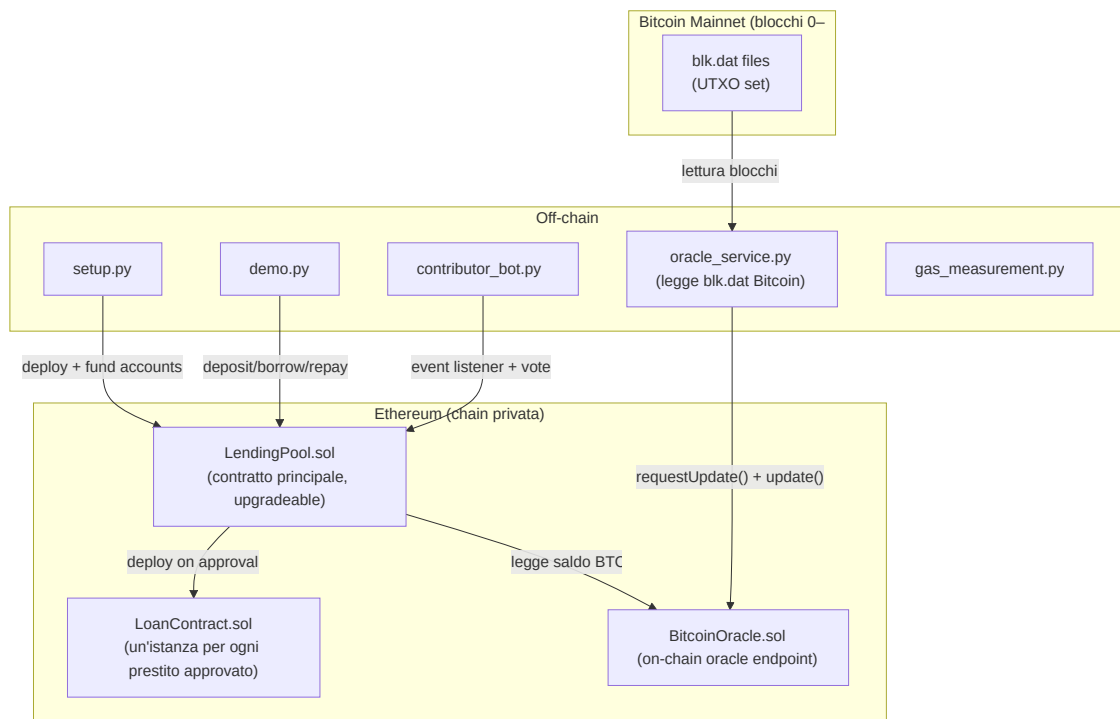


Fig. — Architettura complessiva del sistema: i contratti Ethereum interagiscono con l'oracle on-chain, aggiornato dall'oracle Python che legge i dati Bitcoin.

Attori e Ruoli

Contributor

Qualsiasi utente con un saldo deposito **non zero** nel funding pool (compresi i fondi attualmente bloccati in prestiti attivi). Può depositare, prelevare, votare proposte di prestito e richiedere compensazioni per prestiti falliti.

Applicant

Qualsiasi utente con una proposta di prestito attiva, un prestito attivo, o un prestito approvato/rifiutato. Lo stesso utente può essere sia contributor che applicant contemporaneamente.

Disposable Value

Per ogni contributor, la quota del suo deposito nel funding pool che **non è attualmente bloccata** in alcun prestito attivo. È la quantità che conta per il voto e per i prelievi.

Componenti da Implementare in Dettaglio

1. Bitcoin Liquidity Oracle

Il ruolo dell'oracolo è spiegato in [Lezione 9 - Bitcoin](#) per quanto riguarda la struttura degli UTXO, e in [Lezione 13 - \(Lab\) Script Classification e blk.dat](#) per la lettura concreta dei file `blk.dat`. Vedi anche [Lezione 8 - Introduzione alla Blockchain](#) per il concetto generale di oracolo cross-chain.

1.1 Smart Contract On-Chain (`BitcoinOracle.sol`)

Il contratto on-chain deve:

- Mantenere una mapping `btcAddress → balance` (in satoshi o BTC) degli indirizzi Bitcoin.
- Esporre una funzione `requestUpdate(string btcAddress) payable` che accetta una **fee** $\geq$ `minOracleFee` ($=$ `gas_cost_of_update * 0.1 gwei`). Questo registra la richiesta.
- Esporre una funzione `update(string btcAddress, uint256 balance)` chiamabile **solo dall'oracle operator** (l'indirizzo del servizio Python) per aggiornare il saldo.
- Esporre una funzione `getBalance(string btcAddress) view returns (uint256)` letta dal LendingPool durante la risoluzione della proposta.
- Emettere un evento `UpdateRequested(string btcAddress, address requester)` sulla chiamata a `requestUpdate`.
- Emettere un evento `BalanceUpdated(string btcAddress, uint256 newBalance)` sulla chiamata a `update`.

Calcolo della minOracleFee

La `minOracleFee` è definita come `gas_cost_of_update_function * 0.1 gwei`. Questo significa che dovrai misurare il gas della funzione `update()` con Hardhat, poi calcolare il valore e impostarlo come costante nel costruttore del contratto (o come variabile immutabile). Fai prima la misura del gas.

1.2 Script Off-Chain (`oracle_service.py`)

Il servizio Python deve:

1. **Connettersi ai file `blk.dat`** del nodo Bitcoin (i primi 131 000 blocchi della mainnet). Si usa la libreria `bitcoin` (o `python-bitcoinlib`) per parsare i blocchi. Vedi [Lezione 13 - \(Lab\) Script Classification e blk.dat](#) per il formato.
2. **Mantenere un UTXO set** aggiornato blocco per blocco:
 - Per ogni transazione in ogni blocco: rimuovere gli input spesi dall'UTXO set, aggiungere gli output non spesi.
 - La coinbase transaction del blocco 0 è un caso speciale.
3. **Ascoltare gli eventi `UpdateRequested`** emessi dal contratto Ethereum (con Web3.py, polling sugli ultimi blocchi).
4. **Calcolare il saldo** dell'indirizzo BTC richiesto sommando tutti i suoi UTXO non spesi.
5. **Chiamare `update()`** sul contratto con il saldo calcolato.

Requisito fondamentale

Il codice deve essere scritto come se elaborasse **un blocco alla volta** (loop su blocchi in ordine crescente), anche se in pratica processa tutti i 131 000 in sequenza all'avvio. Questo simula il comportamento real-time di un nodo Bitcoin.

Limite dei blocchi

Processare **solo i primi 131 000 blocchi** della mainnet Bitcoin, non tutta la blockchain. Questo è per contenere i tempi di sync e le dimensioni dei file.

2. Smart Contracts del Lending Service

Vedi [Lezione 20 - \(Lab\) Solidity](#) per la sintassi base di Solidity, [Lezione 21 - \(Lab\) Solidity Avanzato](#) per pattern avanzati, e [Lezione 23 - \(Lab\) Advanced Solidity - Vulnerabilities e Upgrading](#) per upgradeability e vulnerabilità. I concetti di gas e transaction sono trattati in [Lezione 19 - Ethereum Accounts Transactions Gas](#).

2.1 `LendingPool.sol` — Contratto Principale

Questo è il contratto centrale che gestisce tutta la logica. Deve essere **upgradeable** (usa il pattern UUPS o Transparent Proxy di OpenZeppelin).

Stato interno:

```

// Pool
mapping(address => uint256) contributorDeposit; // deposito
totale per contributor
mapping(address => uint256) contributorLocked; // valore
bloccato in prestiti attivi
uint256 totalFundingPool;
uint256 compensationPool;

// Proposte
mapping(uint256 => LoanProposal) proposals;
uint256 proposalCount;

// Collateral percentage (globale, aggiornato da esito
prestiti)
uint256 currentCollateralPercentage; // iniziale: 50, range [1,
100]

// Costanti
uint256 constant PROPOSAL_VOTING_PERIOD = 12; // in block
height diff
uint256 constant MIN_DEPOSIT = 100_000; // wei
uint256 constant INITIAL_COLLATERAL = 50; // %
uint256 constant BTC_ETH_RATE = 30; // 1 BTC =
30 ETH
address immutable bitcoinOracleAddress;

```

Struttura `LoanProposal`:

```

struct LoanProposal {
    address applicant;
    uint256 amount;
    uint8 interestRate; // 1-100
    uint256 duration; // in block height diff
    string btcAddress;
    uint256 submittedAtBlock;
    bool resolved;
    bool approved;
    mapping(address => bool) hasVoted;
    mapping(address => bool) voteValue; // true = approve

```

```

    address[] voters;
}

```

Operazioni per Contributor:

- `deposit() payable` — deposita ETH. Richiede `msg.value >= MIN_DEPOSIT`. Aggiorna `contributorDeposit[msg.sender]` e `totalFundingPool`.
- `withdraw(uint256 amount)` — preleva ETH. `amount` deve essere $\leq \text{disposableValue}(\text{msg.sender}) = \text{contributorDeposit}[\text{msg.sender}] - \text{contributorLocked}[\text{msg.sender}]$. Aggiorna mappings e trasferisce ETH.
- `vote(uint256 proposalId, bool approve)` — vota su una proposta attiva (non ancora risolta). Può votare solo chi è contributor. Un contributor può votare una sola volta per proposta.
- `requestCompensation(uint256 loanId)` — richiede compensazione per un prestito fallito (vedi sezione dedicata).

Operazioni per Applicant:

- `requestOracleUpdate(string btcAddress) payable` — inoltro della fee all'oracolo. Chiama `BitcoinOracle.requestUpdate{value: msg.value}(btcAddress)`.
- `submitProposal(uint256 amount, uint8 interestRate, uint256 duration, string btcAddress)` — crea una nuova proposta. Emette `ProposalSubmitted(uint256 proposalId, address applicant, uint256 amount)`.
- `resolveProposal(uint256 proposalId)` — risolve una proposta (vedi logica sotto).
- `repayLoan(uint256 loanId) payable` — rimborso parziale o totale (vedi logica sotto).

Logica di risoluzione proposta:

```

function resolveProposal(uint256 proposalId) {
    // Solo l'applicant originale può risolvere
    require(msg.sender == proposal.applicant);
    // Solo dopo il voting period
    require(block.number >= proposal.submittedAtBlock +
PROPOSAL_VOTING_PERIOD);
    // Non già risolta
}

```

```

require(!proposal.resolved);

// Check 1: liquidità nel pool
uint256 totalDisposable = computeTotalDisposable();
if (totalDisposable < proposal.amount) {
    → reject (close proposal)
}

// Check 2: Bitcoin liquidity
uint256 btcBalance =
oracle.getBalance(proposal.btcAddress);
uint256 btcInEth = btcBalance * BTC_ETH_RATE; // BTC→ETH
conversion
if (btcInEth < proposal.amount) {
    → reject (close proposal)
}

// Count weighted votes
uint256 weightedApprove = 0;
uint256 weightedTotal = totalDisposable;
for each voter who voted approve:
    weightedApprove += disposableValue(voter_i)
// Note: non-voters count as reject

if (weightedApprove * 2 > weightedTotal) { // strict
majority
    → approve: lock funds, deploy LoanContract
} else {
    → reject (tie or majority reject)
}
}

```

Logica di lock dei fondi (se approvato):

Il valore viene bloccato proporzionalmente al `disposableValue` di ogni contributor. Per ogni contributor `i`:

$$\text{locked}_i = \left\lfloor \frac{\text{loan_amount} \times \text{disposable}_i}{\text{totalDisposable}} \right\rfloor$$

L'eventuale scarto per aritmetica intera viene **sottratto dall'importo del prestito** (non accreditato all'applicant). Il contratto `LoanContract` viene deployato con l'importo effettivo (somma dei `locked_i`). Il collateral percentage corrente al momento dell'approvazione viene fissato nel `LoanContract`.

☰ Esempio di lock proporzionale

Contributor 1: disposable = 10, Contributor 2: disposable = 20, Contributor 3: disposable = 30. Totale = 60. Loan = 12.

- $\text{locked}_1 = \text{floor}(12 * 10 / 60) = \text{floor}(2.0) = 2$
- $\text{locked}_2 = \text{floor}(12 * 20 / 60) = \text{floor}(4.0) = 4$
- $\text{locked}_3 = \text{floor}(12 * 30 / 60) = \text{floor}(6.0) = 6$
- Totale locked = 12, scarto = 0. Importo effettivo = 12.

2.2 `LoanContract.sol` — Contratto per Singolo Prestito

Deployato dal `LendingPool` per ogni prestito approvato. Gestisce tutto il ciclo di vita del singolo prestito.

Stato interno:

```
address applicant;
uint256 loanAmount;           // importo effettivo (dopo
arrotondamento)
uint8 interestRate;
uint256 duration;             // in blocchi
uint256 startBlock;
uint8 collateralPercentage;    // fisso al momento
dell'approvazione
uint256 totalRepaid;          // quanto già rimborsato
bool failed;
bool successful;

// Partecipanti ordinati per valore locked (dal più alto al più
basso)
// tie-break: ordine crescente di indirizzo
struct Participant {
    address addr;
```



```

uint256 lockedAmount;
uint256 repaidAmount; // quanto del base amount è già
stato rimborsato
}
Participant[] participants; // ordinati per lockedAmount desc,
addr asc

address lendingPool; // per le callbacks di
aggiornamento stato

```

Logica di rimborso parziale (`partialRepay()` payable):

Solo l'applicant originale può chiamare questa funzione. La somma totale da rimborsare per completare il prestito è `loanAmount + loanAmount * interestRate / 100`.

La divisione di `msg.value` avviene in questo ordine:

1. **Interesse** = `min(msg.value, totalInterestDue * fraction_repaid_this_call)`

- In pratica, ogni pagamento viene prima attribuito alla quota di interessi proporzionale.
- Formula semplificata: se `msg.value <= interestDueRemaining` → tutto è interesse; altrimenti la parte eccedente il debito residuo di interesse è base amount.

⚠ Ordine corretto di attribuzione

Il modo più robusto è: calcola quanto resta da rimborsare in totale (`loanAmount + interest - totalRepaid`). La quota di interesse di questo pagamento è `payment * interestRate / (100 + interestRate)` (oppure calcola proporzionalmente sul dovuto residuo). Verifica la formula con esempi numerici prima di implementarla.

2. **Base amount**: rimborsato ai contributor **in ordine** (highest locked → lowest, tie-break address asc). Il contributor viene rimborsato fino al suo `lockedAmount`. Il LendingPool aggiorna `contributorDeposit` e `contributorLocked` dopo ogni rimborso.
3. **Eccesso** (se `totalRepaid > loanAmount`): va al compensation pool.
4. **Interesse** → **gain + collateral**:

- $gain = interest * (100 - collateralPercentage) / 100$ → distribuito ai contributor con la stessa regola di ordinamento (proporzionale al `lockedAmount` originale). Arrotondamenti → compensation pool.
- $collateral = interest * collateralPercentage / 100$ → tutto al compensation pool.

5. Se dopo questo pagamento `totalRepaid >= loanAmount`: prestito **successful**. Callback al LendingPool per aggiornare `currentCollateralPercentage` (-5, min 1).

Logica della compensazione per prestito fallito:

Un prestito è **failed** se `totalRepaid < loanAmount` AND `block.number > startBlock + duration`. La funzione `requestCompensation(address contributor)` può essere chiamata da qualsiasi contributor che abbia fondi bloccati nel prestito:

- Calcola il danno residuo del contributor: `lockedAmount_i - repaidAmount_i`.
- Preleva dal compensation pool (del LendingPool) il minimo tra il danno residuo e il disponibile.
- Trasferisce al contributor.
- Aggiorna `repaidAmount_i` per riflettere la compensazione (così futuri rimborsi parziali da parte dell'applicant vengono ridistribuiti correttamente).
- Marca il prestito come **failed** la prima volta che viene chiamata (il marking avviene una volta sola; un prestito failed non può diventare successful).
- Callback al LendingPool per aggiornare `currentCollateralPercentage` (+5, max 100).

2.3 Upgradeability (`LendingPoolProxy`)

Vedi [Lezione 23 - \(Lab\) Advanced Solidity - Vulnerabilities e Upgrading](#) per i pattern di upgradeability. Usa il pattern **UUPS** (EIP-1822) con OpenZeppelin:

- `LendingPool.sol` eredita da `UUPSUpgradeable` e `OwnableUpgradeable`.
- Sostituisci il costruttore con `initialize()`.
- Deploy tramite `ERC1967Proxy`.

3. Script Python

3.1 `setup.py` — Configurazione Iniziale

Deve creare i nuovi account necessari (non usare i prefunded accounts per deploy!), trasferire ETH da un prefunded account ai nuovi account, deployare i contratti nell'ordine corretto (prima oracle, poi LendingPool con l'indirizzo dell'oracle), e stampare tutti gli indirizzi dei contratti deployati.

3.2 `demo.py` — Scenario di Esempio

Deve eseguire un workflow completo e stampare i saldi dopo ogni operazione significativa:

1. Deposito di più contributor
2. Richiesta di update Oracle (con indirizzo BTC reale nei primi 131k blocchi)
3. Sottomissione di una proposta di prestito
4. Votazione da parte dei contributor
5. Risoluzione della proposta (approvata)
6. Esecuzione del prestito (trasferimento ETH all'applicant)
7. Rimborso parziale
8. Rimborso finale
9. Stampa stato pool, collateral percentage, compensation pool

3.3 `contributor_bot.py` — Strategia Automatica

Il bot deve:

- Ascoltare gli eventi `ProposalSubmitted` emessi dal LendingPool (polling con Web3.py).
- Per ogni nuova proposta, votare `approve` (anche se il contributor ha tutti i fondi bloccati — il voto viene castato comunque).
- Stampare ogni voto emesso con i dettagli della proposta.

3.4 `gas_measurement.py`

Script (o test Hardhat) per misurare il gas di ogni operazione:

- `deposit()`, `withdraw()`, `vote()`, `submitProposal()`, `resolveProposal()`, `partialRepay()`, `requestCompensation()`, `requestUpdate()` (oracle), `update()` (oracle).

4. Test Hardhat

Vedi [Lezione 20 - \(Lab\) Solidity](#) e [Lezione 21 - \(Lab\) Solidity Avanzato](#) per l'uso di Hardhat con Ethers.js. I test devono coprire:

- Deposit e withdraw (happy path + edge cases: sotto minimum, oltre disposable).
- Proposal submission e voting.
- Risoluzione con approvazione e con rifiuto (per liquidità insufficiente, BTC check fallito, maggioranza reject).
- Rimborso parziale e totale con verifica della distribuzione corretta.
- Compensazione per prestito fallito.
- Reentrancy attack (test separato su contratti modificati).
- Upgradeability (deploy proxy, upgrade implementazione).

5. Analisi Vulnerabilità e Strategie Malevole

5.1 Reentrancy Vulnerability

Vedi [Lezione 23 - \(Lab\) Advanced Solidity - Vulnerabilities e Upgrading](#) per i dettagli tecnici. La funzione `withdraw()` è il candidato naturale: invia ETH prima di aggiornare lo stato. Per dimostrare:

1. Modifica `withdraw()` per usare `call{value:}` prima di `contributorDeposit[msg.sender] -= amount`.
2. Crea `MaliciousContributor.sol` con una `receive()` che richiama `withdraw()`.
3. Test Hardhat che dimostra il drain del pool.

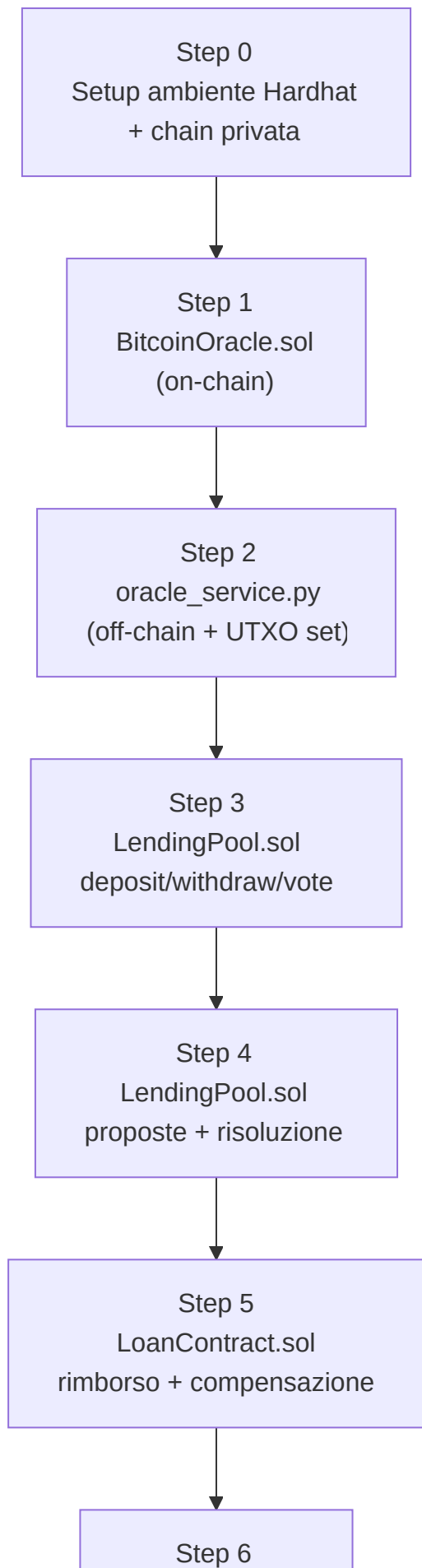
5.2 Strategia Malevola senza Reentrancy

La domanda è: può un contributor disonesto guadagnare più del dovuto sfruttando la regola di rimborso per ordine di locked value?

Possibile strategia: un contributor malevolo deposita un importo molto grande subito prima del lock di un prestito, ottenendo una quota di lock proporzionalmente alta e quindi venendo rimborsato per primo (essendo il contributor con il locked value più alto). Questo non gli dà un vantaggio economico diretto sul base amount, ma lo protegge maggiormente dai default. Tuttavia, il gain degli interessi segue la stessa regola proporzionale, quindi non

c'è un extra gain effettivo. La vera analisi richiede di verificare se il **timing** di deposit/withdraw attorno ai momenti di lock può creare asimmetrie.

Piano di Implementazione Passo per Passo



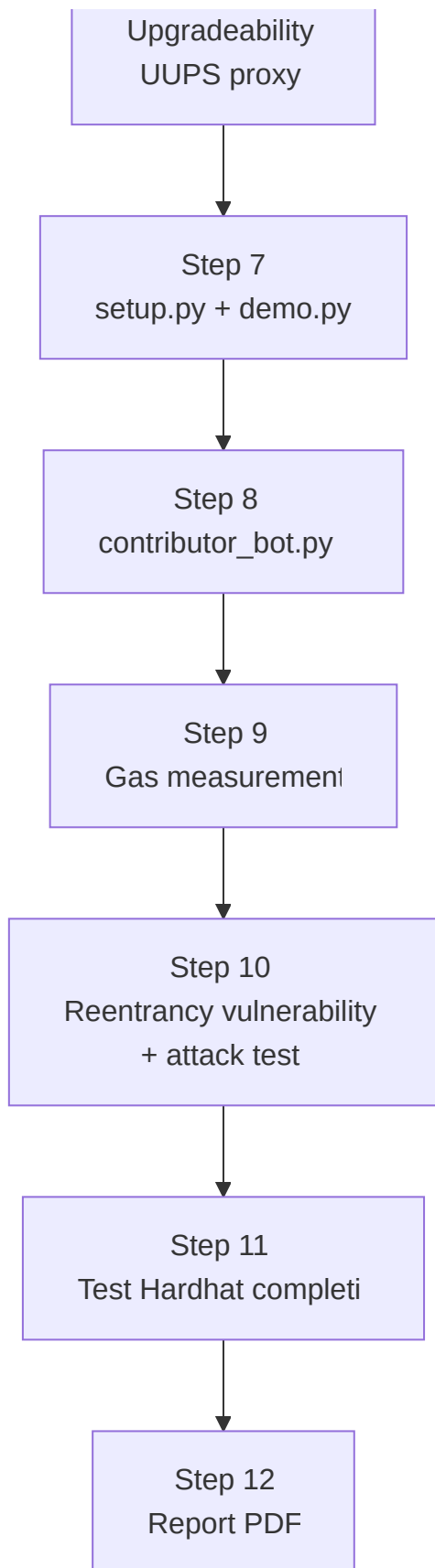


Fig. — Sequenza di implementazione consigliata. Ogni step dipende dai precedenti.

Step 0 — Setup Ambiente

1. Installa Node.js, Hardhat, OpenZeppelin contracts.
2. Crea struttura cartelle: `hardhat/contracts/`, `hardhat/test/`, `python/oracle/`, `python/scripts/`.
3. Configura `hardhat.config.js` per connettersi alla chain privata locale (genesis file fornito dal corso). Porta tipica: 8545.
4. Verifica che la chain parta con `geth --datadir ... init genesis.json && geth --datadir ... --http ...`.
5. Installa dipendenze Python: `web3`, `python-bitcoinlib` (o `bitcoin`).

⚠ Account prefunded

Gli account prefunded nel genesis file possono **solo** trasferire ETH ad altri account. Non possono deployare contratti né eseguire altre transazioni. Usa Web3.py per creare nuovi account (`w3.eth.account.create()`) e finanziati dai prefunded accounts tramite trasferimento.

Step 1 — BitcoinOracle Smart Contract

Implementa `BitcoinOracle.sol`. Scrivi un test Hardhat minimale che:

- Deploya il contratto.
- Chiama `requestUpdate("1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf")` (indirizzo del blocco genesis Bitcoin) con la fee corretta.
- Chiama `update(...)` con un saldo di test.
- Verifica che `getBalance(...)` restituisca il valore corretto.

Misura il gas di `update()` in questo step, perché serve per calcolare `minOracleFee`.

Step 2 — Oracle Off-chain

Implementa `oracle_service.py`. Testa il parsing dei primi blocchi Bitcoin in isolamento prima di integrare con Ethereum. Usa un indirizzo Bitcoin noto (es. l'indirizzo del blocco genesis: `1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf`) per verificare che il UTXO set sia corretto.

Step 3 — LendingPool: Fondamenta

Implementa le operazioni base: `deposit()`, `withdraw()`, `disposableValue()`. Scrivi test che verifichino:

- Deposit sotto minimum → revert.
- Withdraw oltre disposable → revert.
- Deposit + withdraw corretti aggiornano i mappings.

Step 4 — LendingPool: Proposte e Voto

Implementa `submitProposal()`, `vote()`. Testa scenari di voto con contributor multipli.

Step 5 — LendingPool: Risoluzione e LoanContract

Implementa `resolveProposal()` con tutti e tre i controlli (liquidità pool, BTC check, voto ponderato). Implementa il lock proporzionale e il deploy di `LoanContract`. Testa il lock con 3 contributor (esempio del PDF).

Step 6 — LoanContract: Rimborso e Compensazione

Implementa `partialRepay()` con la distribuzione corretta (base amount per ordine, gain proporzionale, collateral al pool). Implementa `requestCompensation()`. Testa rimborsi multipli e parziali.

Step 7 — Upgradeability

Wrappa `LendingPool` in un proxy UUPS. Verifica che un upgrade dell'implementazione mantenga lo stato.

Step 8 — Script Python

Implementa `setup.py`, `demo.py`, `contributor_bot.py`. Testa end-to-end sulla chain privata.

Step 9 — Gas Measurement

Raccoglie i gas cost di tutte le operazioni. Tabella nel report.

Step 10 — Reentrancy

Crea la versione vulnerabile e il contratto attaccante. Test Hardhat.

Step 11 — Test Suite Completa

Completa la copertura dei test Hardhat.

Step 12 — Report

Scrivi il report PDF (max 5 pagine).

Costanti di Riferimento Rapido

Costante	Valore
Proposal voting period	12 blocchi
Minimum deposit	100 000 wei
Initial collateral %	50%
Collateral step	±5% per prestito failed/successful
Collateral range	[1%, 100%]
BTC/ETH rate	1 BTC = 30 ETH
Oracle min fee	gas(update) × 0.1 gwei
Bitcoin blocks da processare	primi 131 000 della mainnet

Prompt per lo Sviluppo con AI

Questa sezione contiene i **prompt completi e pronti all'uso** da dare in sequenza a un'AI (es. Claude) per costruire il progetto. Ogni prompt è autocontenuto: include tutto il contesto necessario per eseguirlo senza riferirsi alla conversazione precedente.

Come usare questi prompt

Incolla ogni prompt in una nuova chat con l'AI, in ordine. Allega i file di output del prompt precedente quando necessario (es. il codice del contratto già scritto). Ogni prompt specifica cosa si aspetta in input e cosa produce in output.

Prompt 0 — Setup del Progetto Hardhat

Sto sviluppando un progetto Ethereum per il corso Peer-to-Peer Systems and Blockchain dell'Università di Pisa (A.A. 2025/26).

Il progetto è un servizio di lending decentralizzato su Ethereum con oracolo Bitcoin.

Crea la struttura di progetto Hardhat completa con le seguenti specifiche:

STRUTTURA CARTELLE:

```

...
project/
├─ hardhat/
│   ├─ contracts/
│   │   ├─ BitcoinOracle.sol
│   │   ├─ LendingPool.sol
│   │   ├─ LoanContract.sol
│   │   └─ MaliciousContributor.sol (placeholder vuoto)
│   └─ test/
│       ├─ Oracle.test.js
│       ├─ LendingPool.test.js
│       ├─ Reentrancy.test.js
│       └─ GasMeasurement.test.js
│   └─ scripts/
│       └─ deploy.js
│   └─ hardhat.config.js
│   └─ package.json
├─ python/
│   ├─ oracle/
│   │   ├─ oracle_service.py
│   │   └─ btc_utils.py
│   ├─ setup.py
│   ├─ demo.py
│   ├─ contributor_bot.py
│   └─ gas_measurement.py
...

```

DIPENDENZE da installare (package.json):

- hardhat ^2.22.0
- @nomicfoundation/hardhat-toolbox
- @openzeppelin/contracts ^5.0.0
- @openzeppelin/contracts-upgradeable ^5.0.0

```
- ethers ^6.0.0
```

hardhat.config.js deve configurare due reti:

1. "hardhat" (default, per i test)
2. "privatechain" con url "http://127.0.0.1:8545" e chainId 12345 (o quello nel genesis file)

Crea tutti i file con il contenuto minimo (placeholder con commenti TODO dove il codice verrà scritto nei prompt successivi). Fornisci:

1. Il package.json completo
2. Il hardhat.config.js completo
3. I file placeholder .sol con SPDX license, versione pragma ^0.8.20, e struttura vuota
4. Il comando npm per installare tutto

Non scrivere ancora la logica dei contratti.

Prompt 1 — BitcoinOracle Smart Contract

Sto sviluppando un servizio di lending decentralizzato su Ethereum. Ho bisogno che tu implementi il smart contract `BitcoinOracle.sol` in Solidity ^0.8.20.

SCOPO: Il contratto è un endpoint on-chain per un oracolo Bitcoin. Memorizza i saldi (in satoshi) degli indirizzi Bitcoin, riceve richieste di aggiornamento a pagamento, e viene aggiornato da un operatore fidato (l'indirizzo del servizio Python off-chain).

SPECIFICHE COMPLETE:

Stato:

- `address public operator` – l'indirizzo dell'oracle service (impostato nel costruttore)
- `uint256 public minFee` – calcolato come `gas_cost_of_update * 0.1 gwei` (passato come parametro al costruttore)
- `mapping(string => uint256) private balances` – indirizzo BTC → saldo in satoshi

- ``mapping(string => bool) private known`` – se l'indirizzo è già stato visto

Funzioni:

1. ``constructor(address _operator, uint256 _minFee)`` – imposta operator e minFee
2. ``requestUpdate(string calldata btcAddress) external payable`` – richiede aggiornamento. Richiede `msg.value >= minFee`. Emette evento `UpdateRequested`. Invia la fee all'operator tramite `call` (non `transfer`).
3. ``update(string calldata btcAddress, uint256 balanceInSatoshi) external`` – solo operator. Aggiorna il saldo. Emette `BalanceUpdated`. Se indirizzo nuovo, imposta `known[btcAddress] = true`.
4. ``getBalance(string calldata btcAddress) external view returns (uint256)`` – restituisce il saldo in satoshi. Se l'indirizzo non è noto, restituisce 0.
5. ``setMinFee(uint256 _minFee) external`` – solo operator. Aggiorna minFee.

Events:

- ``event UpdateRequested(string btcAddress, address indexed requester, uint256 fee)``
- ``event BalanceUpdated(string btcAddress, uint256 newBalanceInSatoshi)``

IMPORTANTE:

- Usa ``require`` con messaggi espliciti per tutti i controlli
- Proteggi ``update()`` e ``setMinFee()`` con ``require(msg.sender == operator, "Only operator")``
- NON usare ``transfer()`` o ``send()`` (usa ``call{value:}`` con controllo di successo)
- Aggiungi un ``receive()`` vuoto al contratto per accettare ETH

Fornisci il file Solidity completo e un test Hardhat (``Oracle.test.js``) che verifichi:

1. Deploy corretto con operator e minFee
2. `requestUpdate` con fee corretta → emette evento
3. `requestUpdate` con fee insufficiente → `revert`

4. update da operator → aggiorna balance
5. update da non-operator → revert
6. getBalance per indirizzo noto e sconosciuto
7. Misura del gas della funzione update() e stampa il valore

Il test deve usare ethers.js v6 e Hardhat.

Prompt 2 — Oracle Off-Chain Python

Sto sviluppando un servizio di lending decentralizzato su Ethereum. Ho bisogno del componente Python off-chain dell'oracolo Bitcoin: uno script che legge i primi 131.000 blocchi della Bitcoin mainnet, mantiene un UTXO set aggiornato, e aggiorna un smart contract Ethereum quando riceve richieste.

CONTRATTO ORACLE (già deployato):

- ABI: { requestUpdate(string btcAddress) payable, update(string btcAddress, uint256 balanceInSatoshi), getBalance(string btcAddress) view returns uint256 }
- Indirizzo: passato come variabile d'ambiente ORACLE_ADDRESS
- Il contratto emette: UpdateRequested(string btcAddress, address requester, uint256 fee)

SPECIFICHE DEL SERVIZIO PYTHON:

File `btc\_utils.py`:

- Funzione `parse\_blocks(blk\_dat\_path: str, max\_blocks: int = 131000)` che usa python-bitcoinlib per parsare i file blk.dat uno alla volta
- Restituisce un generatore di oggetti Block (un blocco alla volta, in ordine)
- Gestisce il magic number e la struttura dei file blk.dat (possono essere blk000000.dat, blk000001.dat, ...)

File `oracle\_service.py`:

- Mantiene un dizionario UTXO set: `{(txid, vout\_index): (address, value\_satoshi)}`
- Per ogni blocco processato (in ordine da 0 a 130.999):
 - Rimuove gli input spesi dall'UTXO set

- Aggiunge gli output non spesi
- Ignora output con script non standard (non P2PKH o P2SH) – registra un warning
- Funzione ``get_address_balance(btc_address: str) -> int`` che somma tutti gli UTXO appartenenti all'indirizzo
- Loop principale:
 1. Carica tutti i 131.000 blocchi una volta all'avvio (costruisce l'UTXO set completo)
 2. Poi entra in un loop di polling (ogni 5 secondi) sugli eventi `UpdateRequested` del contratto Ethereum
 3. Per ogni evento trovato: calcola il saldo, chiama `oracle_contract.update(btcAddress, balance)`
 4. Stampa ogni update eseguito

CONFIGURAZIONE (variabili d'ambiente):

- `BLK_DAT_DIR`: cartella con i file `blk.dat`
- `ETH_RPC_URL`: es. `http://127.0.0.1:8545`
- `ORACLE_ADDRESS`: indirizzo del contratto oracle
- `OPERATOR_PRIVATE_KEY`: chiave privata dell'operator per firmare le transazioni `update()`

DIPENDENZE PYTHON:

- `web3` (`pip install web3`)
- `python-bitcoinlib` (`pip install python-bitcoinlib`)
- `python-dotenv`

IMPORTANTE:

- Il codice deve processare i blocchi uno alla volta (loop, non batch) anche se in pratica li carica tutti sequenzialmente
- Gestisci le eccezioni per blocchi malformati (continua al blocco successivo)
- Stampa il progresso ogni 10.000 blocchi
- La funzione ``update()`` sul contratto deve essere inviata come transazione firmata dall'operator

Fornisci entrambi i file Python completi e funzionanti.

Aggiungi un piccolo script di test ``test_btc_utils.py`` che verifica il parsing del blocco genesis (block 0) e controlla

che l'indirizzo 1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf sia nell'UTXO set con il giusto valore (50 BTC = 5.000.000.000 satoshi).

Prompt 3 — LendingPool: Deposit, Withdraw, Struttura Base

Sto sviluppando `LendingPool.sol`, il contratto principale di un servizio di lending decentralizzato su Ethereum. Questo prompt implementa la struttura base e le operazioni di deposit/withdraw. Userò prompt successivi per aggiungere le proposte, il voto e la risoluzione.

Il contratto deve essere UPGRADEABLE usando il pattern UUPS di OpenZeppelin 5.x.

COSTANTI:

- PROPOSAL_VOTING_PERIOD = 12 (blocchi)
- MIN_DEPOSIT = 100_000 (wei)
- INITIAL_COLLATERAL_PERCENTAGE = 50
- BTC_ETH_RATE = 30 (1 BTC = 30 ETH)

STATO DEL CONTRATTO:

```
```solidity
// Pool
mapping(address => uint256) public contributorDeposit; //
deposito totale
mapping(address => uint256) public contributorLocked; //
bloccato in prestiti
uint256 public totalFundingPool; //
somma di tutti i depositi
uint256 public compensationPool; //
pool di compensazione

// Collateral
uint256 public currentCollateralPercentage; //
parte da interesse → pool
uint256 public totalLoans; //
contatore prestiti totali
uint256 public failedLoans; //
```



```

contatore prestiti falliti
uint256 public successfulLoans;

// Oracle
address public bitcoinOracleAddress;

// Lista contributor (per iterazione durante lock)
address[] public contributors;
mapping(address => bool) public isContributor;
```

```

FUNZIONI DA IMPLEMENTARE:

1. ``initialize(address _oracleAddress) public initializer`` – setup iniziale (UUPS pattern). Imposta oracle, collateral iniziale, owner.
2. ``deposit() public payable`` – deposita ETH.
 - Richiede `msg.value >= MIN_DEPOSIT`
 - Aggiorna `contributorDeposit[msg.sender] += msg.value`
 - Aggiorna `totalFundingPool += msg.value`
 - Se nuovo contributor: aggiunge a `contributors[]`, imposta `isContributor[msg.sender] = true`
 - Emette `Deposited(address contributor, uint256 amount)`
3. ``withdraw(uint256 amount) public`` – preleva ETH.
 - Richiede `amount > 0`
 - Richiede `amount <= disposableValue(msg.sender)`
 - Aggiorna `contributorDeposit[msg.sender] -= amount`
 - Aggiorna `totalFundingPool -= amount`
 - Se `contributorDeposit[msg.sender] == 0`: rimuove da `contributors[]` e imposta `isContributor = false`
 - Trasferisce ETH con `call{value:}`
 - Emette `Withdrawn(address contributor, uint256 amount)`
4. ``disposableValue(address contributor) public view returns (uint256)`` – restituisce `contributorDeposit[contributor] - contributorLocked[contributor]`

5. ``totalDisposableValue() public view returns (uint256)`` – somma di `disposableValue` per tutti i contributor in `contributors[]`

6. ``_authorizeUpgrade(address) internal override onlyOwner`` – richiesto da UUPS

EVENTI:

- `Deposited(address indexed contributor, uint256 amount)`
- `Withdrawn(address indexed contributor, uint256 amount)`

IMPORTANTE:

- Eredita da `UUPSUpgradeable`, `OwnableUpgradeable`
- Nessun constructor, usa `initialize()`
- Usa `ReentrancyGuardUpgradeable` per proteggere `deposit()` e `withdraw()`
- NON usare `transfer()` o `send()` per ETH

Fornisci il contratto Solidity completo per questa parte e un test Hardhat (``LendingPool.test.js``, sezione "Pool Management") che verifichi:

1. Deploy tramite proxy ERC1967
2. Deposit corretto (aggiorna stato, emette evento)
3. Deposit sotto `MIN_DEPOSIT` → revert
4. Withdraw entro `disposable` → corretto
5. Withdraw oltre `disposable` → revert
6. `totalDisposableValue()` con 3 contributor
7. Un contributor che si toglie dal pool dopo withdraw totale

Prompt 4 — LendingPool: Proposte di Prestito e Voting

Continuo lo sviluppo di ``LendingPool.sol``. Ho già implementato `deposit()`, `withdraw()`, e la struttura base (vedi codice allegato). Ora aggiungi la logica delle proposte di prestito e del sistema di voto.

[ALLEGA: `LendingPool.sol` dal prompt precedente]

STRUTTURA DATI DA AGGIUNGERE:

```

```solidity
struct LoanProposal {
 address applicant;
 uint256 amount; // importo richiesto in wei
 uint8 interestRate; // 1-100 (percentuale sul
principal)
 uint256 duration; // in block height difference
 string btcAddress; // indirizzo Bitcoin per il check
liquidità
 uint256 submittedAtBlock; // block.number al momento della
submission
 bool resolved; // true se già risolta (approvata
o rifiutata)
 bool approved; // true se approvata
 address[] voterList; // lista di chi ha votato (per
iterazione)
 mapping(address => bool) hasVoted;
 mapping(address => bool) voteValue; // true = approve,
false = reject
}

mapping(uint256 => LoanProposal) public proposals;
uint256 public proposalCount;
```

```

NOTA: i mapping dentro struct in Solidity non possono essere in storage pubblico. Usa una funzione getter custom.

FUNZIONI DA AGGIUNGERE:

1. `requestOracleUpdate(string calldata btcAddress) external payable``
 - Trasferisce msg.value all'oracle tramite `IBitcoinOracle(bitcoinOracleAddress).requestUpdate{value: msg.value}(btcAddress)`
 - Emette `OracleUpdateRequested(address applicant, string btcAddress)`

2. ``submitProposal(uint256 amount, uint8 interestRate, uint256 duration, string calldata btcAddress) external returns (uint256 proposalId)``

- Richiede interestRate tra 1 e 100
- Richiede amount > 0
- Richiede duration > 0
- Crea nuova LoanProposal con submittedAtBlock = block.number
- Restituisce il proposalId (incrementa proposalCount)
- Emette ProposalSubmitted(uint256 indexed proposalId, address indexed applicant, uint256 amount, uint8 interestRate, uint256 duration, string btcAddress)

3. ``vote(uint256 proposalId, bool approve) external``

- Richiede che il voter sia contributor (isContributor[msg.sender] == true)
- Richiede che la proposta esista e non sia risolta
- Richiede che il voter non abbia già votato
- Registra il voto in hasVoted e voteValue
- Aggiunge msg.sender a voterList
- Emette VoteCast(uint256 indexed proposalId, address indexed voter, bool approve)

4. Getter helper:

- ``getProposal(uint256 proposalId) external view returns (address applicant, uint256 amount, uint8 interestRate, uint256 duration, string memory btcAddress, uint256 submittedAtBlock, bool resolved, bool approved)``
- ``hasVoted(uint256 proposalId, address voter) external view returns (bool)``
- ``getVote(uint256 proposalId, address voter) external view returns (bool)``

INTERFACCIA ORACLE (aggiungi):

```
```solidity
```

```
interface IBitcoinOracle {
```

```
 function requestUpdate(string calldata btcAddress) external payable;
```

```
 function getBalance(string calldata btcAddress) external
```

```
view returns (uint256);
}
```

```

EVENTI:

- OracleUpdateRequested(address indexed applicant, string btcAddress)
- ProposalSubmitted(uint256 indexed proposalId, address indexed applicant, uint256 amount, uint8 interestRate, uint256 duration, string btcAddress)
- VoteCast(uint256 indexed proposalId, address indexed voter, bool approve)

Fornisci il LendingPool.sol aggiornato (solo le parti aggiunte/modificate, con commento // NUOVO per le sezioni aggiunte) e aggiungi al test file la sezione "Loan Proposals and Voting" con test per:

1. submitProposal con parametri validi → emette evento corretto
2. submitProposal con interestRate=0 → revert
3. vote da contributor → registra voto corretto
4. vote da non-contributor → revert
5. doppio voto → revert
6. voto su proposta inesistente → revert

Prompt 5 — LendingPool: Risoluzione Proposta e Deploy LoanContract

Continuo lo sviluppo di `LendingPool.sol` e implemento `LoanContract.sol`. Questa è la parte più complessa: la risoluzione delle proposte di prestito con voto ponderato, il lock proporzionale dei fondi, e il deploy del contratto per il singolo prestito.

[ALLEGA: LendingPool.sol aggiornato e LoanContract.sol placeholder]

LOGICA DI RISOLUZIONE (da aggiungere a LendingPool.sol):

```
```solidity
```

```

function resolveProposal(uint256 proposalId) external {
 LoanProposal storage prop = proposals[proposalId];

 // Check: solo l'applicant originale
 require(msg.sender == prop.applicant, "Only applicant");
 // Check: non già risolta
 require(!prop.resolved, "Already resolved");
 // Check: voting period trascorso
 require(block.number >= prop.submittedAtBlock +
PROPOSAL_VOTING_PERIOD, "Voting period not over");

 prop.resolved = true;

 // Check 1: liquidità nel pool
 uint256 totalDisp = totalDisposableValue();
 if (totalDisp < prop.amount) {
 emit ProposalRejected(proposalId, "Insufficient pool
liquidity");
 return;
 }

 // Check 2: Bitcoin liquidity check
 uint256 btcBalance =
IBitcoinOracle(bitcoinOracleAddress).getBalance(prop.btcAddress
);
 // btcBalance è in satoshi. Converti: 1 BTC = 1e8 satoshi =
30 ETH = 30e18 wei
 // quindi btcBalanceInWei = btcBalance * 30e18 / 1e8 =
btcBalance * 3e11
 uint256 btcInWei = btcBalance * 30e18 / 1e8;
 if (btcInWei < prop.amount) {
 emit ProposalRejected(proposalId, "BTC liquidity check
failed");
 return;
 }

 // Count weighted votes (approve only, reject = default)
 uint256 weightedApprove = 0;
 for (uint i = 0; i < prop.voterList.length; i++) {

```

```

 address voter = prop.voterList[i];
 if (prop.voteValue[voter]) { // voted approve
 weightedApprove += disposableValue(voter);
 }
 }
 // strict majority: weightedApprove > totalDisp / 2
 // equivalente a: weightedApprove * 2 > totalDisp (evita
divisione)
 if (weightedApprove * 2 <= totalDisp) {
 emit ProposalRejected(proposalId, "Majority reject");
 return;
 }

 // APPROVATO: lock fondi proporzionalmente e deploy
LoanContract
 prop.approved = true;
 _lockAndDeployLoan(proposalId);
}
```

```

FUNZIONE `\_lockAndDeployLoan(uint256 proposalId) internal`:

```

```
uint256 totalDisp = totalDisposableValue();
uint256 actualLoanAmount = 0;
address[] memory participants = new address[]
(contributors.length);
uint256[] memory lockedAmounts = new uint256[]
(contributors.length);
uint256 count = 0;

for each contributor in contributors[:
 if disposableValue(contributor) == 0: skip
 uint256 share = floor(prop.amount *
disposableValue(contributor) / totalDisp)
 if share == 0: skip
 contributorLocked[contributor] += share
 participants[count] = contributor
 lockedAmounts[count] = share
```

```

```

        actualLoanAmount += share
        count++

// deploy LoanContract
LoanContract loan = new LoanContract(
    prop.applicant,
    actualLoanAmount,
    prop.interestRate,
    prop.duration,
    currentCollateralPercentage,
    participants[0..count],
    lockedAmounts[0..count],
    address(this) // lendingPool callback
);

// transfer ETH to loan contract (che poi lo trasferisce
// all'applicant)
(bool ok,) = address(loan).call{value: actualLoanAmount}("");
require(ok, "Transfer to loan failed");

emit ProposalApproved(proposalId, address(loan),
    actualLoanAmount);
`

```

IMPLEMENTA `LoanContract.sol`:

Costruttore: riceve tutti i parametri e memorizza i participant ordinati per lockedAmount DESC, tie-break per indirizzo ASC. Nell'ordine di lock: priorità rimborso del base amount.

Dopo il deploy, il costruttore trasferisce immediatamente l'actualLoanAmount all'applicant.

Stato interno:

```

`
solidity
address public applicant;
address public lendingPool;
uint256 public loanAmount;           // importo effettivo
uint8 public interestRate;

```



```

uint256 public duration;           // in blocchi
uint256 public startBlock;         // block.number al deploy
uint8 public collateralPercentage;
uint256 public totalRepaid;
bool public isFailed;
bool public isSuccessful;
bool public failedMarked;          // il marking del failed
avviene solo una volta

struct Participant {
    address addr;
    uint256 lockedAmount;           // al momento del lock
    uint256 baseRepaid;             // quanto del base è già stato
    restituito
    uint256 compensationPaid; // quanto compensazione già
    ricevuta
}
Participant[] public participants; // ordinati per lockedAmount
DESC, addr ASC (tie-break)
```

```

Funzione `isExpired() public view returns (bool)` =  
 block.number > startBlock + duration  
 Funzione `isFailed() public view returns (bool)` =  
 isExpiredImpl && totalRepaid < loanAmount && !isSuccessful

#### EVENTI:

- ProposalRejected(uint256 indexed proposalId, string reason)
- ProposalApproved(uint256 indexed proposalId, address  
loanContract, uint256 actualAmount)
- LoanDeployed(address indexed loanContract, address indexed  
applicant, uint256 amount)

Fornisci LendingPool.sol (parti modificate) e LoanContract.sol  
 completo fino al costruttore con ordering. Aggiungi test per:

1. Resolve troppo presto → revert
2. Resolve con liquidità insufficiente → ProposalRejected
3. Resolve con BTC check fallito → ProposalRejected (mocka  
l'oracle)

4. Resolve con majority reject → ProposalRejected
5. Resolve approvato → ProposalApproved, LoanContract deployato, ETH trasferito all'applicant
6. Verifica lock proporzionale con 3 contributor (10/20/30, loan=12)

## Prompt 6 — LoanContract: Rimborso e Compensazione

Implemento la logica di rimborso e compensazione in `LoanContract.sol`. Ho già il contratto con costruttore e strutture dati. [ALLEGA: LoanContract.sol e LendingPool.sol correnti]

FUNZIONE `partialRepay() external payable` (solo applicant):

Il totale da rimborsare è:  $\text{loanAmount} + \text{loanAmount} * \text{interestRate} / 100 = \text{loanAmount} * (100 + \text{interestRate}) / 100$

Ogni volta che viene chiamata con msg.value:

1. Calcola quanta parte è "interesse" e quanta è "base amount":
  - $\text{totalDue} = \text{loanAmount} * (100 + \text{interestRate}) / 100$
  - $\text{alreadyRepaid} = \text{totalRepaid}$
  - $\text{remaining} = \text{totalDue} - \text{alreadyRepaid}$  (se già tutto rimborsato, revert o return)
  - $\text{payment} = \text{msg.value}$  (capped a remaining se eccede)
  - Per separare base e interesse in questo pagamento:
    - $\text{baseRemaining} = \text{loanAmount} - \min(\text{totalRepaid basePortion accumulata}, \text{loanAmount})$
    - Il pagamento viene applicato prima al base amount, poi all'interesse...

ATTENZIONE: usa la formula corretta. Il modo più robusto:

- $\text{interestTotal} = \text{loanAmount} * \text{interestRate} / 100$
- $\text{baseRepaidSoFar}$  = quanto del loanAmount è già stato rimborsato nel base
- $\text{interestRepaidSoFar}$  = quanto di interestTotal è già stato rimborsato
- In questo pagamento, prima si rimborsa il base rimanente, poi l'interesse rimanente.

```

- `baseThisPayment = min(payment, loanAmount -
baseRepaidSoFar)`
- `interestThisPayment = min(payment - baseThisPayment,
interestTotal - interestRepaidSoFar)`
- `excessThisPayment = payment - baseThisPayment -
interestThisPayment` (va al compensation pool)

2. Distribuisci `baseThisPayment` ai participant in ordine
(highest lockedAmount first, tie-break addr asc):
- Per ogni participant[i]:
 - rimborsoable = lockedAmount[i] - baseRepaid[i]
 - creditato = min(baseThisPayment_remaining, rimborsoable)
 - chiama
LendingPool.creditContributorBase(participant[i].addr,
creditato)
 - baseRepaid[i] += creditato
 - baseThisPayment_remaining -= creditato

3. Distribuisci `interestThisPayment`:
- gain = interestThisPayment * (100 - collateralPercentage)
/ 100
- collateral = interestThisPayment - gain (il resto va tutto
al comp pool)
- Chiama LendingPool.addToCompensationPool(collateral)
- Distribuisci gain ai participant proporzionalmente al
lockedAmount originale (stessa regola ordinamento):
- Per ogni participant[i]:
 - share = floor(gain * lockedAmount[i] / loanAmount)
 - chiama
LendingPool.creditContributorGain(participant[i].addr, share)
- Qualsiasi resto (arrotondamento) va al compensation pool

4. Aggiorna totalRepaid += baseThisPayment +
interestThisPayment

5. Rimanda al mittente l'eccesso oltre totalDue se msg.value >
remaining

6. Se dopo questo pagamento baseRepaidSoFar >= loanAmount:

```

```

- isSuccessful = true
- Chiama LendingPool.onLoanSuccess() → aggiorna
currentCollateralPercentage -= 5 (min 1)
- Chiama LendingPool.unlockFunds(participants,
lockedAmounts) per sbloccare contributorLocked
- Emette LoanRepaid(address applicant, uint256 totalPaid)

FUNZIONI CALLBACK nel LendingPool (da aggiungere):
- `creditContributorBase(address contributor, uint256 amount)
external onlyLoanContract`
 → contributorDeposit[contributor] += amount; (il base amount
torna nel deposito)
 → contributorLocked[contributor] -= amount (si sblocca)
 → totalFundingPool += amount
- `creditContributorGain(address contributor, uint256 amount)
external onlyLoanContract`
 → Trasferisce ETH direttamente al contributor (non nel
funding pool)
- `addToCompensationPool(uint256 amount) external
onlyLoanContract`
 → compensationPool += amount
- `onLoanSuccess() external onlyLoanContract`
 → successfulLoans++; aggiorna currentCollateralPercentage
- `onLoanFailed() external onlyLoanContract`
 → failedLoans++; aggiorna currentCollateralPercentage (+5,
max 100), chiamata una sola volta per prestito
- `unlockFunds(address[] calldata contributors, uint256[]
calldata amounts) external onlyLoanContract`
 → per ogni i: contributorLocked[contributors[i]] -=
amounts[i]

```

```

FUNZIONE `requestCompensation() external` (in LoanContract,
chiamabile da contributor):
- Richiede che il prestito sia failed (isExpired &&
!isSuccessful)
- Trova participant corrispondente a msg.sender
- danno = lockedAmount[i] - baseRepaid[i] - compensationPaid[i]
- Se danno <= 0: revert ("Nothing to compensate")
- Chiede a LendingPool di erogare dal compensation pool:

```

```
LendingPool.payCompensation(msg.sender, danno)
- Aggiorna compensationPaid[i] con quanto effettivamente
ricevuto
- Se prima volta che questo loan viene marcato failed: chiama
LendingPool.onLoanFailed()
- Emette CompensationPaid(address contributor, uint256 amount)
```

```
FUNZIONE LendingPool `payCompensation(address contributor,
uint256 requested) external onlyLoanContract returns (uint256
paid)`:
```

```
- paid = min(requested, compensationPool)
- compensationPool -= paid
- Trasferisce paid a contributor
- Restituisce paid
```

```
MODIFICATORE `onlyLoanContract` nel LendingPool:
```

```
- Tiene traccia dei loan contract deployati: `mapping(address
=> bool) public isRegisteredLoan`
- Impostato a true quando il loan viene deployato in
`_lockAndDeployLoan`
```

Fornisci:

1. LoanContract.sol completo con partialRepay() e requestCompensation()
2. Aggiornamenti a LendingPool.sol (solo le nuove funzioni)
3. Test per:
  - Rimborso totale in un colpo solo (verifica distribuzione base e gain)
  - Rimborso in 3 rate (verifica stato intermedio)
  - Rimborso oltre il dovuto (eccesso al compensation pool)
  - Compensazione su prestito fallito (dopo scadenza)
  - Compensazione con compensation pool insufficiente (rimborso parziale)
  - Chiamata a requestCompensation su prestito non fallito → revert

## Prompt 7 — Upgradeability UUPS

Implemento il pattern di upgradeability UUPS per  
``LendingPool.sol``. [ALLEGA: `LendingPool.sol` completo]

Devo:

1. Verificare che `LendingPool` erediti da `UUPSUpgradeable` e `OwnableUpgradeable` (OpenZeppelin 5.x)
2. Rimuovere qualsiasi constructor e usare ``initialize()`` con ``initializer`` modifier
3. Implementare ``_authorizeUpgrade(address newImplementation)`` internal override `onlyOwner {}``
4. Creare un contratto ``LendingPoolV2.sol`` di test (aggiunge solo una funzione ``version() returns (string)`` che ritorna "v2") per dimostrare l'upgrade
5. Scrivere il deploy script (``scripts/deploy.js``) che:
  - Deploya `BitcoinOracle`
  - Deploya `LendingPool` tramite `ERC1967Proxy` (il proxy chiama ``initialize(oracleAddress)``)
  - Stampa gli indirizzi di tutti i contratti
6. Scrivere un test Hardhat che:
  - Deploya `LendingPool V1` tramite proxy
  - Verifica che funzioni correttamente (`deposit`)
  - Fa upgrade a `LendingPoolV2` tramite ``upgrades.upgradeProxy`` di `hardhat-upgrades`
  - Verifica che lo stato (depositi esistenti) sia preservato dopo l'upgrade
  - Verifica che la nuova funzione ``version()`` sia disponibile

Usa ``@openzeppelin/hardhat-upgrades`` plugin. Aggiorna ``hardhat.config.js`` per includerlo.

**IMPORTANTE:** In OpenZeppelin 5.x, le variabili di storage nelle classi upgradeable devono usare ERC7201 namespaced storage per evitare collisioni durante gli upgrade. Mostra come implementarlo correttamente oppure usa la struttura classica con le dovute avvertenze.

Fornisci:

1. `LendingPool.sol` (versione finale con UUPS)
2. `LendingPoolV2.sol` (versione di test)

3. scripts/deploy.js aggiornato
4. Test di upgradeability

## Prompt 8 — Script Python: setup.py e demo.py

Implemento gli script Python di setup e demo per il servizio di lending decentralizzato su Ethereum.

CONTRATTI (ABI disponibili nella cartella hardhat/artifacts/):

- LendingPool (proxy upgradeable) a indirizzo

LENDING\_POOL\_ADDRESS

- BitcoinOracle a indirizzo ORACLE\_ADDRESS

CONFIGURAZIONE (da .env):

- ETH\_RPC\_URL=http://127.0.0.1:8545
- GENESIS\_PREFUNDED\_KEY=<chiave privata account prefunded dal genesis>
- LENDING\_POOL\_ADDRESS=<indirizzo dopo deploy>
- ORACLE\_ADDRESS=<indirizzo oracle>
- ORACLE\_OPERATOR\_KEY=<chiave privata dell'operator oracle>

setup.py – Configurazione iniziale:

1. Connettiti alla chain con web3.py
2. Crea 5 nuovi account (w3.eth.account.create())
  - account\_contributor\_1, account\_contributor\_2, account\_contributor\_3
  - account\_applicant\_1
  - account\_oracle\_operator (corrisponde all'operator del BitcoinOracle)
3. Trasferisci ETH da un prefunded account a ciascuno (es. 10 ETH ciascuno)
 

IMPORTANTE: il prefunded account non può deployare contratti, ma può trasferire ETH
4. Stampa tutti gli indirizzi e le private keys (per copiarle nel .env)
5. Salva le chiavi in un file `accounts.json` (NON in produzione, solo per test)
6. Opzionalmente: deploya i contratti se non sono già stati deployati (carica gli ABI da artifacts/)

demo.py – Scenario completo:

1. Carica gli account da accounts.json
2. Stampa saldi iniziali di tutti gli account e del pool
3. contributor\_1 deposita 1 ETH
4. contributor\_2 deposita 2 ETH
5. contributor\_3 deposita 3 ETH
6. Stampa stato del pool (totalFundingPool, disposableValue per ciascuno)
7. applicant\_1 richiede update oracle per l'indirizzo "1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf" (con fee corretta)
8. applicant\_1 sottomete proposta: amount=1 ETH, interestRate=10, duration=100 blocchi, btcAddress="1A1zP1eP5QGefi2DMPTfTL5SLmv7Divf"
9. Stampa proposalId
10. contributor\_1 vota approve
11. contributor\_2 vota approve
12. contributor\_3 vota approve
13. Avanza di 12+ blocchi (usa hardhat\_mine JSON-RPC o mina blocchi vuoti)
14. applicant\_1 richiede risoluzione
15. Stampa stato: proposta approvata?, indirizzo LoanContract
16. Stampa saldi (applicant\_1 dovrebbe avere +1 ETH circa)
17. applicant\_1 ripaga metà del dovuto ( $\text{loanAmount} * (100 + \text{interestRate}) / 100 / 2$ )
18. Stampa stato: totalRepaid, isSuccessful
19. applicant\_1 ripaga il resto
20. Stampa stato finale: isSuccessful=true, compensation pool, saldi contributor

#### IMPORTANTE:

- Dopo ogni operazione significativa stampa: [STEP N]

Descrizione operazione

- Poi stampa i saldi rilevanti
- Usa w3.eth.send\_raw\_transaction con account firmante
- Gestisci la stima del gas (w3.eth.estimate\_gas)
- Stampa il transaction hash per ogni operazione



Fornisci entrambi i file Python completi e funzionanti con gestione degli errori.

## Prompt 9 — contributor\_bot.py

Implemento `contributor\_bot.py`, uno script Python che simula un contributor automatico che vota sempre "approve" su ogni nuova proposta di prestito.

CONTRATTO: LendingPool (proxy upgradeable) – ABI da artifacts/  
CONFIGURAZIONE (da .env o accounts.json):

- ETH\_RPC\_URL
- LENDING\_POOL\_ADDRESS
- BOT\_CONTRIBUTOR\_PRIVATE\_KEY (il contributor che fa il bot)

COMPORTAMENTO DEL BOT:

1. All'avvio: verifica che l'account del bot sia un contributor (abbia un deposito > 0). Se non lo è, stampa un warning ma continua (il bot tenta di votare comunque, e il voto fallirà – che è il comportamento atteso secondo le specifiche).
2. Polling degli eventi ProposalSubmitted dal contratto ogni 3 secondi.
3. Per ogni nuova proposta trovata (proposalId non già processato):
  - a. Stampa: [BOT] Nuova proposta rilevata: proposalId={id}, amount={amount}, applicant={addr}
  - b. Verifica se il bot ha già votato (chiama hasVoted(proposalId, botAddress))
  - c. Se non ha votato: chiama vote(proposalId, true) → approve
  - d. Stampa: [BOT] Voto APPROVE emesso per proposta {id} | tx: {txhash}
  - e. Se il voto fallisce (es. non è contributor): stampa [BOT] Voto non riuscito per proposta {id}: {errore}
4. Mantiene un set di proposalId già processati per evitare doppi voti.
5. Loop infinito con gestione di Ctrl+C (graceful exit).

EVENTI DA ASCOLTARE:

- ProposalSubmitted(uint256 indexed proposalId, address indexed

```
applicant, uint256 amount, ...)
```

#### IMPORTANTE:

- Usa il polling dei log con `w3.eth.get\_logs()` con filtro sull'evento
- Mantieni l'ultimo blocco processato per non riprocessare eventi già visti
- Il bot funziona anche se tutti i fondi del contributor sono bloccati (il voto viene emesso ma potrebbe essere di peso zero – come da specifica)

Fornisci lo script Python completo con commenti esplicativi sulle parti non ovvie.

## Prompt 10 — Gas Measurement

Implemento la misurazione del gas per tutte le operazioni del servizio di lending. [ALLEGA: tutti i contratti .sol e i test esistenti]

Crea un file `test/GasMeasurement.test.js` in Hardhat che misura il gas di ogni operazione e stampa una tabella riassuntiva.

#### OPERAZIONI DA MISURARE:

1. BitcoinOracle.requestUpdate(btcAddress) – con fee corretta
2. BitcoinOracle.update(btcAddress, balance) – l'operazione chiave per la minFee
3. LendingPool.deposit() – con MIN\_DEPOSIT
4. LendingPool.withdraw(amount) – importo tipico
5. LendingPool.submitProposal(...) – proposta standard
6. LendingPool.vote(proposalId, true) – voto singolo
7. LendingPool.resolveProposal(proposalId) – con 3 contributor votanti
8. LoanContract.partialRepay() – rimborso parziale al 50%
9. LoanContract.partialRepay() – rimborso totale in un'unica chiamata
10. LoanContract.requestCompensation() – su prestito failed

FORMATO OUTPUT:

```

'''
=====
|| Operazione || Gas Used || Costo @10
|| gwei || || ||
=====
|| BitcoinOracle.update() || 45,231 || 0.000452
|| ETH || || ||
|| ... || ... || ...
|| || ||
=====
'''

```

Per ciascuna operazione:

- Esegui la transazione
- Usa ``receipt.gasUsed`` per ottenere il gas
- Calcola il costo in ETH a 10 gwei di gas price

Alla fine stampa anche il valore calcolato della ``minOracleFee``  
`= gasUsed(update) * 0.1 gwei.`

Fornisci il file di test completo. Il test non deve fare assert  
(non è un test di correttezza), ma solo misurare e stampare.

Poi crea anche uno script Python equivalente

``python/gas_measurement.py`` che fa la stessa cosa interagendo  
con la chain privata reale (non con Hardhat simulate).

## Prompt 11 — Vulnerabilità Reentrancy e Attacco

Implemento la dimostrazione della vulnerabilità di reentrancy  
nel contratto `LendingPool`. Vedi il tutorial del corso per il  
pattern standard.

[ALLEGA: `LendingPool.sol` completo]

STEP 1: Crea `contracts/LendingPoolVulnerable.sol`  
 Copia LendingPool e modifica la funzione `withdraw()` per introdurre la vulnerabilità:

- PRIMA trasferisce ETH all'utente (con `call{value:}`)
- POI aggiorna `contributorDeposit[msg.sender] -= amount`

La versione vulnerabile deve avere:

```

` ``solidity
function withdraw(uint256 amount) public nonReentrant_DISABLED
{
 require(amount > 0);
 require(amount <= disposableValue(msg.sender), "Exceeds
disposable");
 // VULNERABILE: aggiorna lo stato DOPO il trasferimento
 (bool ok,) = msg.sender.call{value: amount}("");
 require(ok, "Transfer failed");
 contributorDeposit[msg.sender] -= amount;
 totalFundingPool -= amount;
 emit Withdrawn(msg.sender, amount);
}
` ``

```

(Nota: `nonReentrant\_DISABLED` è solo un placeholder per documentare che la guard è rimossa intenzionalmente – non è un vero modificatore)

STEP 2: Crea `contracts/MaliciousContributor.sol`

Un contratto attaccante che:

1. Ha una funzione `attack(address lendingPool, uint256 depositAmount) external payable`
2. Deposita `depositAmount` nel pool vulnerabile
3. Chiama `withdraw(depositAmount)` una prima volta
4. Nella `receive()` (triggered dal trasferimento ETH):
  - Se il balance del lendingPool è ancora > depositAmount: chiama di nuovo `withdraw(depositAmount)`
  - Altrimenti: smette di richiamare (evita revert per fondi insufficienti)
5. Dopo l'attacco: ha prelevato molto più di `depositAmount` dal pool

STEP 3: Crea ``test/Reentrancy.test.js`` con:

- Setup: 3 victim contributors depositano 2 ETH ciascuno nel pool vulnerabile
- Attacker deposita 0.1 ETH
- Attacker esegue l'attacco
- Verifica: il pool è svuotato (o quasi), l'attacker ha guadagnato più di quanto depositato
- Stampa balance prima e dopo per mostrare l'impatto
- Mostra anche che sul contratto NON vulnerabile (con ReentrancyGuard) lo stesso attacco fallisce

IMPORTANTE:

- Il test deve compilare ed eseguire con ``npx hardhat test``
- Commenta il codice dell'attaccante per spiegare ogni passo

Fornisci tutti e tre i file completi.

## Prompt 12 — Hardhat Test Suite Completa

Completa la suite di test Hardhat per il progetto di lending decentralizzato. Ho già i test parziali da prompt precedenti.  
[ALLEGA: tutti i file `test/*.test.js` esistenti e tutti i contratti]

Crea o integra in ``test/LendingPool.test.js`` i seguenti scenari mancanti:

SCENARIO A: Prestito fallito e compensazione

1. 3 contributor depositano (1 ETH, 2 ETH, 3 ETH)
2. Applicant propone prestito da 6 ETH, duration=10 blocchi
3. Tutti votano approve
4. Avanza di 12 blocchi (`hardhat_mine`)
5. Applicant risolve → approvato
6. Avanza di 10 blocchi (loan scaduto)
7. Verificare che `isExpired() == true`
8. `contributor_1` richiede compensazione
9. Verifica che `contributor_1` abbia ricevuto ETH dal `compensation pool`
10. Verifica che il prestito sia marcato come failed

#### SCENARIO B: Collateral percentage dinamico

1. Avvia con `currentCollateralPercentage` = 50
2. Esegui un prestito `successful` (rimborso totale) → verifica che scenda a 45
3. Esegui un prestito `failed` (compensazione) → verifica che salga a 50
4. Esegui 9 prestiti `failed` di fila → verifica che si attesti a 95 (max 100)

#### SCENARIO C: Proposta rifiutata per liquidità insufficiente

1. contributor deposita 0.5 ETH
2. Applicant propone prestito da 1 ETH
3. Tutti votano `approve`
4. Avanza 12 blocchi
5. `Resolve` → rifiutato (pool insufficiente)
6. Verifica emissione evento `ProposalRejected` con ragione corretta

#### SCENARIO D: Voto ponderato – tie e majority reject

1. `contributor_1` deposita 10 ETH, `contributor_2` deposita 10 ETH
2. `contributor_1` vota `approve`, `contributor_2` vota `reject`
3. `Resolve` → rifiutato (tie → reject)

#### SCENARIO E: Rimborso con distribuzione corretta (verifiche numeriche)

Setup: `contributor_1`=10 ETH, `contributor_2`=20 ETH; `loan`=6 ETH, `interestRate`=10, `collateral`=50

Dopo lock: `locked_1`=2, `locked_2`=4, `actualLoan`=6

Rimborso: 6.6 ETH (6 base + 0.6 interest)

- `interest`=0.6 ETH → `gain`=0.3 ETH (50%), `collateral`=0.3 ETH
- `gain` distribuito: `contributor_1` riceve `floor(0.3 * 2/6)`=0.1 ETH, `contributor_2` riceve `floor(0.3 * 4/6)`=0.2 ETH
- `compensationPool` += 0.3 (collateral) + 0 (rounding su gain)
- `contributor_1.deposit` ripristinato di 2 ETH, `contributor_2` di 4 ETH

Scrivi `assertions` numeriche per ogni valore.

Fornisci il file di test completo e integrato con tutti gli

scenari. Ogni test deve essere indipendente (usa `beforeEach` con `fixture`).

## Prompt 13 — Analisi Strategia Malevola e Report

Aiutami a completare l'analisi della strategia malevola e la struttura del report PDF per il progetto di lending decentralizzato.

DOMANDA DEL PROGETTO:

"Discuss if there is a possible malicious strategy for a contributor to gain an unfair remuneration at the expense of other honest contributors (without relying on a reentrancy attack), e.g., by leveraging the loan repayment majority rule."

ANALISI RICHIESTA:

Analizza queste possibili strategie e concludi se sono possibili/efficaci:

1. **\*\*Front-running del lock\*\***: un contributor malevolo deposita un importo enorme appena prima che una proposta venga risolta (in modo da ottenere la quota maggiore di lock e quindi essere rimborsato per primo dal base amount). È fattibile su una chain privata senza mempool ordering? Quali sono i costi/benefici?
2. **\*\*Manipolazione dell'ordinamento di rimborso\*\***: il base amount viene rimborsato in ordine decrescente di locked value. Un contributor con il locked value più alto riceve i suoi fondi indietro prima degli altri in caso di rimborso parziale. In un prestito che va parzialmente male, questo contributor perde meno. Mostra un esempio numerico.
3. **\*\*Strategia di voto strategico\*\***: un contributor può depositare, aspettare una proposta rischiosa, votare reject, e poi depositare di più per proposte buone. Questo non è "unfair" per definizione, ma analizza se il timing del deposito/ritiro attorno ai voti può creare asimmetrie.
4. **\*\*Cycling attack\*\***: deposita → vota approve su una proposta

propria (da un altro account) → ottieni il prestito → non ripaghi → aspetta la compensazione → ripeti. Analizza se questo è limitato dalla struttura del sistema.

Per la prima strategia implementa un test Hardhat dimostrativo che mostra l'effetto del front-running sull'ordinamento del rimborso.

STRUTTURA DEL REPORT (5 pagine max):

Sezione 1 – Scelte implementative (1.5 pp):

- Architettura: perché UUPS vs Transparent Proxy
- Gestione dell'ordinamento dei participant nel LoanContract
- Gestione aritmetica intera per evitare rounding attacks
- Come è implementato il "one-time failed marking"
- Come è gestita la callback LoanContract → LendingPool (sicurezza onlyLoanContract)

Sezione 2 – Gas Evaluation (0.5 pp):

- Tabella con i risultati del gas\_measurement
- Valore calcolato della minOracleFee
- Operazione più costosa e motivazione

Sezione 3 – Reentrancy (1 pp):

- Descrizione della vulnerabilità introdotta
- Come funziona l'attacco (con pseudocodice o snippet)
- Riferimento al test dimostrativo

Sezione 4 – Strategia malevola (1 pp):

- Analisi delle strategie trovate
- Conclusione: quale è più efficace? È difendibile?

Sezione 5 – Manuale utente (1 pp):

- Come avviare la chain privata
- Come eseguire setup.py
- Come avviare oracle\_service.py
- Come eseguire demo.py
- Come eseguire i test Hardhat
- Struttura delle cartelle consegnate



Fornisci:

1. L'analisi completa della strategia malevola (testo in italiano, da copiare nel report)
2. Il test Hardhat per il front-running (se dimostrabile)
3. La struttura dettagliata del report in formato Markdown (da convertire in PDF)

## Consegna — Checklist Finale

### 🔗 Cosa va consegnato su Moodle

- ☐ Tutti i file `.sol` dei smart contracts
- ☐ Cartella `hardhat/` completa (contracts, test, scripts, config, node\_modules opzionale o .zip del progetto)
- ☐ `python/setup.py`, `python/demo.py`, `python/contributor_bot.py`
- ☐ `python/oracle/oracle_service.py` e `python/oracle/btc_utils.py`
- ☐ `python/gas_measurement.py`
- ☐ Test suite completa (file `.test.js`)
- ☐ Report PDF (max 5 pagine)
- ☐ Tutto in un unico `.zip` con struttura di cartelle intelligibile

### ⚠️ Vincoli importanti da non dimenticare

- Gli account prefunded del genesis possono **solo trasferire ETH**, non deployare contratti.
- Ogni `LoanContract` deve essere deployato come **contratto separato** (non come struttura dati nel LendingPool).
- Il "failed marking" avviene **una volta sola** per prestito e non è reversibile.
- Il collateral percentage rimane nel range `[1, 100]`.
- Tutti i periodi di tempo sono in **block height difference**, non in secondi.
- Il prestito viene concesso con l'importo **effettivo** (dopo arrotondamento), non quello proposto.

### ✍️ Gruppi

Il progetto può essere svolto da **al massimo 2 studenti**. Ciascuno deve essere in grado di discuterlo da solo all'orale. L'esame orale comprende sia la

discussione del progetto che domande sugli argomenti delle lezioni.